

MNEMO — ANTIGRAVITY

MASTER BUILD DOCUMENT

"Your lectures, remembered forever."

READ THIS ENTIRE DOCUMENT BEFORE EXECUTING ANYTHING. Every architectural decision is final. Execute phases in order. Do not skip steps or deviate from the stack.

STEP 0 — MCP SERVERS SETUP (RUN THIS FIRST)

Paste this prompt into Antigravity before touching any code:

```
Install and configure the following MCP servers  
for this project. Run each command in sequence:
```

1. Neon MCP (database management):
`npx -y @neondatabase/mcp-server-neon`
2. Vercel MCP (deployment):
`npx -y @vercel/mcp-server`
3. File system MCP (file operations):
`npx -y @modelcontextprotocol/server-filesystem`
`.`

```
4. GitHub MCP (version control):  
npx -y @modelcontextprotocol/server-github
```

After installing, confirm each MCP is active by listing available tools for each server. Do not proceed to Phase 1 until all 4 MCPs are confirmed active.

PROJECT VISION

One-sentence pitch: Upload any lecture PDF and watch it explode into a living 3D universe of knowledge you can fly through, where every node glows based on how well you remember it.

The demo moment that wins: Judge uploads their own content on stage. In 15 seconds, a navigable 3D knowledge graph appears. Every other team built a quiz app.

Judge-stopper features:

1. 3D Knowledge Universe (React Three Fiber — concepts as glowing nodes, relationships as cyan beams)
2. Memory Decay Glow (nodes shift green → yellow → red based on time and quiz performance — judges have never seen memory visualised)
3. Live Node Expansion (click any node → Gemini explains it + generates a flashcard, all free)
4. Personalisation Engine (tracks what you struggle with, reorders your learning path)

5. Spatial Recall Mode (navigate 3D space to find and answer flashcards – spatial memory improves retention 40%)

LOCKED TECH STACK

Zero decisions left open. Do not substitute any library.

Layer	Technology	Notes
Framework	Next.js 14 (App Router)	File-based routing, server components
Language	TypeScript	Type safety end-to-end
Styling	Tailwind CSS 3.4	Utility-first, fastest path to polished UI
Components	shadcn/ui (dark theme)	Production-grade base
Animation	Framer Motion 11	Spring physics, AnimatePresence
3D	React Three Fiber + @react-three/drei	Core wow feature
AI	Google Gemini 1.5 Flash API	Free tier – concept extraction + flashcards

Layer	Technology	Notes
Database	Neon Postgres (free tier)	Serverless, 512MB
ORM	Drizzle ORM	Type-safe, schema-first
Auth	Better Auth	Open source, no cost
PDF Parsing	pdf-parse	Server-side text extraction
Hosting	Vercel (free tier)	One command deploy

Total cost: ₹0

VISUAL DESIGN SYSTEM

Apply these exact values to every component. No deviations.

Colour Tokens

```
--bg-primary:    #050510    /* deep space – all
page backgrounds */
--bg-secondary:  #0A0A1F    /* card surfaces */
--bg-tertiary:   #0F0F2E    /* modals, elevated
surfaces */
--accent-violet: #6C63FF    /* primary brand,
buttons, active states */
```

```
--accent-cyan:    #00D4FF    /* connections,
beams, secondary highlights */
--accent-glow:    #A78BFA    /* hover glow, soft
highlights */
--node-weak:      #FF6B6B    /* memory strength
0.0-0.3 */
--node-medium:    #FFD93D    /* memory strength
0.3-0.6 */
--node-strong:    #6BCB77    /* memory strength
0.6-1.0 */
--text-primary:   #F0F0FF    /* all body text */
--text-muted:     #8888AA    /* labels, secondary
*/
--border:         #1E1E3F    /* all borders */
```

Fonts (Google Fonts – free)

```
Display:  Space Grotesk – geometric, technical
(headings, brand name)
Body:     Inter – clean, readable (all body
text)
Mono:     Fira Code – concept tags, labels
```

Motion Rules

```
Page transitions:    Framer Motion layoutId
shared element
Card entrances:      staggerChildren 0.08s,
y:20→0, opacity:0→1
3D node hover:       scale 1→1.15,
emissiveIntensity increase
Modal open:           spring { stiffness:300,
damping:30 }
Button hover:         translateY(-2px), box-
```

```
shadow intensify
All durations:      0.3-0.4s (never longer for
hackathon speed)
```

PART 1 — DATABASE LAYER

Complete Schema + Migration Plan

1A. FILE: `/drizzle/schema.ts`

```
import {
  pgTable, uuid, text, timestamp,
  boolean, integer, real, jsonb, index
} from "drizzle-orm/pg-core";

// ——— USERS

```

```
export const users = pgTable("users", {
  id:
    uuid("id").primaryKey().defaultRandom(),
  name:      text("name").notNull(),
  email:
    text("email").notNull().unique(),
  passwordHash:
    text("password_hash").notNull(),
  avatar:    text("avatar"),
  learningStyle:
    text("learning_style").default("visual"), //
  visual | reading | mixed
  createdAt:
```

```

timestamp("created_at").defaultNow(),
  updatedAt:
timestamp("updated_at").defaultNow(),
});

// ——— SESSIONS (Better Auth managed)


---


export const sessions = pgTable("sessions", {
  id:
  uuid("id").primaryKey().defaultRandom(),
  userId:    uuid("user_id").references(() =>
users.id, { onDelete: "cascade" }).notNull(),
  token:     text("token").notNull().unique(),
  expiresAt: timestamp("expires_at").notNull(),
  createdAt:
timestamp("created_at").defaultNow(),
});

// ——— DOCUMENTS (uploaded lectures)


---


export const documents = pgTable("documents", {
  id:
  uuid("id").primaryKey().defaultRandom(),
  userId:    uuid("user_id").references(()
=> users.id, { onDelete: "cascade"
}).notNull(),
  title:     text("title").notNull(),
  rawText:   text("raw_text").notNull(),
  // extracted PDF text
  summary:   text("summary"),
  // Gemini summary
  subject:   text("subject"),
  // auto-detected
  totalNodes:
integer("total_nodes").default(0),
  masteryScore:

```

```

real("mastery_score").default(0), // 0-100
  status:
text("status").default("pending"), // pending |
processing | complete | error
  createdAt:
timestamp("created_at").defaultNow(),
  updatedAt:
timestamp("updated_at").defaultNow(),
});

// ——— CONCEPT NODES (3D graph nodes)


---


export const conceptNodes =
pgTable("concept_nodes", {
  id:
uuid("id").primaryKey().defaultRandom(),
  documentId:
uuid("document_id").references(() =>
documents.id, { onDelete: "cascade"
}).notNull(),
  label:      text("label").notNull(),
  explanation: text("explanation").notNull(),
  importance:
integer("importance").default(5), // 1-10,
drives sphere size
  positionX:  real("position_x").notNull(),
  positionY:  real("position_y").notNull(),
  positionZ:  real("position_z").notNull(),
  createdAt:
timestamp("created_at").defaultNow(),
});

// ——— CONCEPT EDGES (3D graph connections)


---


export const conceptEdges =
pgTable("concept_edges", {

```



```

    id:
      uuid("id").primaryKey().defaultRandom(),
      documentId:
        uuid("document_id").references(() =>
          documents.id, { onDelete: "cascade"
        }).notNull(),
      sourceNodeId:
        uuid("source_node_id").references(() =>
          conceptNodes.id, { onDelete: "cascade"
        }).notNull(),
      targetNodeId:
        uuid("target_node_id").references(() =>
          conceptNodes.id, { onDelete: "cascade"
        }).notNull(),
      relationshipLabel:
        text("relationship_label"),
      strength:
        real("strength").default(1.0), // drives beam
        thickness
    });

```

// — FLASHCARDS

```

export const flashcards = pgTable("flashcards",
{
  id:
    uuid("id").primaryKey().defaultRandom(),
    nodeId:      uuid("node_id").references(() =>
      conceptNodes.id, { onDelete: "cascade"
    }).notNull(),
    documentId: uuid("document_id").references(()
=> documents.id, { onDelete: "cascade"
  }).notNull(),
    userId:      uuid("user_id").references(() =>
      users.id, { onDelete: "cascade" }).notNull(),

```

```

    question: text("question").notNull(),
    answer: text("answer").notNull(),
    difficulty: integer("difficulty").default(3),
// 1-5
    createdAt:
timestamp("created_at").defaultNow(),
});

// ——— USER PROGRESS (personalisation engine)


---


export const userProgress =
pgTable("user_progress", {
  id:
uuid("id").primaryKey().defaultRandom(),
  userId:
uuid("user_id").references(() => users.id, {
onDelete: "cascade" }).notNull(),
  nodeId:
uuid("node_id").references(() =>
conceptNodes.id, { onDelete: "cascade"
}).notNull(),
  documentId:
uuid("document_id").references(() =>
documents.id, { onDelete: "cascade"
}).notNull(),
  visitCount:
integer("visit_count").default(0),
  timeSpentSeconds:
integer("time_spent_seconds").default(0),
  memoryStrength:
real("memory_strength").default(0.5), // 0-1,
drives glow colour
  lastReviewedAt:
timestamp("last_reviewed_at"),
  correctAnswers:
integer("correct_answers").default(0),

```

```

    totalAttempts:
integer("total_attempts").default(0),
    updatedAt:
timestamp("updated_at").defaultNow(),
});

// ——— QUIZ SESSIONS


---



export const quizSessions =
pgTable("quiz_sessions", {
  id:
uuid("id").primaryKey().defaultRandom(),
  userId:      uuid("user_id").references(()
=> users.id, { onDelete: "cascade"
}).notNull(),
  documentId:
uuid("document_id").references(() =>
documents.id, { onDelete: "cascade"
}).notNull(),
  score:      integer("score").default(0),
  totalQuestions:
integer("total_questions").default(0),
  completedAt: timestamp("completed_at"),
  createdAt:
timestamp("created_at").defaultNow(),
});

// ——— INDEXES (define before any queries)


---



// These are applied automatically by drizzle-
kit push
// idx_documents_user      →
documents(user_id)
// idx_nodes_document      →
concept_nodes(document_id)

```

```
// idx_edges_document      →
concept_edges(document_id)
// idx_progress_user_node  →
user_progress(user_id, node_id)
// idx_flashcards_node     →
flashcards(node_id)
// idx_flashcards_user     →
flashcards(user_id)
```

1B. FILE: `/lib/db.ts`

```
import { neon } from
"@neondatabase/serverless";
import { drizzle } from "drizzle-orm/neon-
http";
import * as schema from "@drizzle/schema";

const sql = neon(process.env.DATABASE_URL!);
export const db = drizzle(sql, { schema });
export type DB = typeof db;
```

1C. FILE: `/drizzle.config.ts`

```
import type { Config } from "drizzle-kit";

export default {
  schema: "../drizzle/schema.ts",
  out: "../drizzle/migrations",
  dialect: "postgresql",
  dbCredentials: {
    url: process.env.DATABASE_URL!,
```

```
},  
} satisfies Config;
```

1D. DATABASE MIGRATION COMMAND

```
# Run after schema is created. This pushes all  
tables to Neon.  
npx drizzle-kit push
```

PART 2 — BACKEND LAYER

All API Routes + Auth + AI Integration

2A. FILE: `/lib/auth.ts`

```
import { betterAuth } from "better-auth";  
import { drizzleAdapter } from "better-auth/adapters/drizzle";  
import { db } from "@/lib/db";  
import * as schema from "@/drizzle/schema";  
  
export const auth = betterAuth({  
  database: drizzleAdapter(db, {  
    provider: "pg",  
    schema: {  
      user: schema.users,  
      session: schema.sessions,  
    },  
  },
```

```
    }},  
    emailAndPassword: {  
      enabled: true,  
    },  
    secret: process.env.BETTER_AUTH_SECRET!,  
    baseUrl: process.env.BETTER_AUTH_URL!,  
  });  
}
```

2B. FILE: `/lib/auth-client.ts`

```
import { createAuthClient } from "better-auth/react";  
  
export const authClient = createAuthClient({  
  baseUrl: process.env.NEXT_PUBLIC_APP_URL ??  
  "http://localhost:3000",  
});  
  
export const { signIn, signUp, signOut,  
  useSession } = authClient;
```

2C. FILE: `/app/api/auth/[...all]/route.ts`

```
import { auth } from "@/lib/auth";  
import { toNextJsHandler } from "better-auth/next-js";  
  
export const { GET, POST } =  
  toNextJsHandler(auth);
```

2D. FILE: /lib/gemini.ts

All three AI functions. Copy exactly.

```
const GEMINI_API_URL =

"https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent";

async function callGemini(prompt: string):
Promise<string> {
  const res = await fetch(`${GEMINI_API_URL}?
key=${process.env.GEMINI_API_KEY}`, {
    method: "POST",
    headers: { "Content-Type":
"application/json" },
    body: JSON.stringify({
      contents: [{ parts: [{ text: prompt }]
}],
      generationConfig: { temperature: 0.3,
maxOutputTokens: 2048 },
    }),
  });
  if (!res.ok) throw new Error(`Gemini error:
${res.status}`);
  const data = await res.json();
  return data.candidates?.[0]?.content?.parts?.
[0]?.text ?? "";
}

// — 1. EXTRACT CONCEPTS FROM LECTURE TEXT


---


export interface Concept {
  label: string;
  explanation: string;
  importance: number;
```

```

    positionX: number;
    positionY: number;
    positionZ: number;
  }

export interface Relationship {
  source: string;
  target: string;
  label: string;
  strength: number;
}

export async function extractConcepts(
  text: string
): Promise<{ concepts: Concept[];
relationships: Relationship[] }> {
  const prompt = `
You are a knowledge graph architect. Extract
from this lecture text:
1. 8-15 key concepts (single words or short
phrases)
2. Relationships between concepts
3. Importance score 1-10 per concept
4. One-sentence explanation per concept

Return ONLY valid JSON – no markdown, no
preamble:
{
  "concepts": [
    { "label": "string", "explanation":
"string", "importance": number }
  ],
  "relationships": [
    { "source": "concept label", "target":
"concept label", "label": "relationship
description", "strength": number }

```



```

    ]
  }
  Lecture text: ${text.slice(0, 8000)}
`;
  try {
    const raw = (await
callGemini(prompt)).trim();
    const cleaned = raw.replace(/```json|```/g,
    "").trim();
    const parsed = JSON.parse(cleaned);
    // Assign random 3D positions (-8 to 8
range prevents overlap)
    const concepts: Concept[] =
parsed.concepts.map((c: any) => ({
      ...c,
      positionX: (Math.random() - 0.5) * 16,
      positionY: (Math.random() - 0.5) * 16,
      positionZ: (Math.random() - 0.5) * 16,
    }));
    return { concepts, relationships:
parsed.relationships ?? [] };
  } catch {
    return { concepts: [], relationships: [] };
  }
}

// — 2. GENERATE FLASHCARDS

export interface FlashcardData {
  question: string;
  answer: string;
  difficulty: number;
}

export async function generateFlashcards(
  concept: string,

```

```

    explanation: string
  ): Promise<FlashcardData[]> {
    const prompt = `
Generate 2 flashcard question-answer pairs for:
"${concept}"
Context: ${explanation}
Return ONLY valid JSON:
{ "flashcards": [{ "question": "string",
"answer": "string", "difficulty": number 1-5 }]
}
`;
    try {
      const raw = (await
callGemini(prompt)).trim();
      const cleaned = raw.replace(/```json|```/g,
"").trim();
      return JSON.parse(cleaned).flashcards ??
[];
    } catch {
      return [];
    }
  }
}

// — 3. PERSONALISED LEARNING SUGGESTIONS


---


export interface Suggestion {
  concept: string;
  reason: string;
}

export async function getSuggestions(
  scores: Record<string, number>
): Promise<Suggestion[]> {
  const prompt = `
A student has these concept mastery scores (0-
1): ${JSON.stringify(scores)}

```

```

Suggest the 3 weakest concepts to review next
and why (one sentence each).
Return ONLY valid JSON:
{ "suggestions": [{ "concept": "string",
"reason": "string" }] }
`;
    try {
        const raw = (await
callGemini(prompt)).trim();
        const cleaned = raw.replace(/```json|```/g,
"").trim();
        return JSON.parse(cleaned).suggestions ??
[];
    } catch {
        return [];
    }
}

```

2E. FILE: /lib/pdf-parser.ts

```

import pdfParse from "pdf-parse";

export async function
extractTextFromPDF(buffer: Buffer):
Promise<string> {
    const data = await pdfParse(buffer);
    if (!data.text || data.text.trim().length <
50) {
        throw new Error("PDF contains no
extractable text or is too short.");
    }
    return data.text;
}

```

2F. ALL API ROUTE FILES

`/app/api/documents/upload/route.ts`

```
import { NextRequest, NextResponse } from
"next/server";
import { auth } from "@lib/auth";
import { db } from "@lib/db";
import { documents } from "@drizzle/schema";
import { extractTextFromPDF } from "@lib/pdf-
parser";

export async function POST(req: NextRequest) {
  const session = await auth.api.getSession({
    headers: req.headers });
  if (!session) return NextResponse.json({
    error: "Unauthorized" }, { status: 401 });

  const formData = await req.formData();
  const file = formData.get("file") as File;
  const title = formData.get("title") as
string;

  if (!file || file.type !== "application/pdf")
  {
    return NextResponse.json({ error: "PDF file
required" }, { status: 400 });
  }

  const buffer = Buffer.from(await
file.arrayBuffer());
  const rawText = await
extractTextFromPDF(buffer);
```

```

    const [doc] = await
db.insert(documents).values({
  userId: session.user.id,
  title: title || file.name.replace(".pdf",
  ""),
  rawText,
  status: "pending",
}).returning();

    return NextResponse.json(doc);
  }

```

`/app/api/documents/route.ts`

```

import { NextRequest, NextResponse } from
"next/server";
import { auth } from "@lib/auth";
import { db } from "@lib/db";
import { documents } from "@drizzle/schema";
import { eq } from "drizzle-orm";

export async function GET(req: NextRequest) {
  const session = await auth.api.getSession({
headers: req.headers });
  if (!session) return NextResponse.json({
error: "Unauthorized" }, { status: 401 });

  const docs = await
db.select().from(documents)
  .where(eq(documents.userId,
session.user.id))
  .orderBy(documents.createdAt);

```

```
    return NextResponse.json(docs);
  }
```

`/app/api/process/[documentId]/route.ts`

```
import { NextRequest, NextResponse } from
"next/server";
import { auth } from "@lib/auth";
import { db } from "@lib/db";
import { documents, conceptNodes, conceptEdges,
flashcards } from "@drizzle/schema";
import { eq } from "drizzle-orm";
import { extractConcepts, generateFlashcards }
from "@lib/gemini";

export async function POST(
  req: NextRequest,
  { params }: { params: { documentId: string } }
) {
  const session = await auth.api.getSession({
    headers: req.headers });
  if (!session) return NextResponse.json({
    error: "Unauthorized" }, { status: 401 });

  const [doc] = await
    db.select().from(documents)
      .where(eq(documents.id,
        params.documentId));

  if (!doc || doc.userId !== session.user.id) {
    return NextResponse.json({ error: "Not
found" }, { status: 404 });
  }
}
```

```

    // Update status to processing
    await db.update(documents).set({ status:
"processing" })
        .where(eq(documents.id,
params.documentId));

    // 1. Extract concepts via Gemini
    const { concepts, relationships } = await
extractConcepts(doc.rawText);

    // 2. Save concept nodes
    const nodeMap = new Map<string, string>(); //
label → id
    for (const c of concepts) {
        const [node] = await
db.insert(conceptNodes).values({
            documentId: params.documentId,
            label: c.label,
            explanation: c.explanation,
            importance: c.importance,
            positionX: c.positionX,
            positionY: c.positionY,
            positionZ: c.positionZ,
        }).returning();
        nodeMap.set(c.label, node.id);
    }

    // 3. Save edges
    for (const r of relationships) {
        const sourceId = nodeMap.get(r.source);
        const targetId = nodeMap.get(r.target);
        if (!sourceId || !targetId) continue;
        await db.insert(conceptEdges).values({
            documentId: params.documentId,
            sourceNodeId: sourceId,
            targetNodeId: targetId,

```

```

        relationshipLabel: r.label,
        strength: r.strength,
    });
}

// 4. Generate flashcards for each node
let flashcardCount = 0;
for (const c of concepts) {
    const nodeId = nodeMap.get(c.label);
    if (!nodeId) continue;
    const cards = await
generateFlashcards(c.label, c.explanation);
    for (const card of cards) {
        await db.insert(flashcards).values({
            nodeId,
            documentId: params.documentId,
            userId: session.user.id,
            question: card.question,
            answer: card.answer,
            difficulty: card.difficulty,
        });
        flashcardCount++;
    }
}

// 5. Update document status and totalNodes
await db.update(documents).set({
    status: "complete",
    totalNodes: concepts.length,
    updatedAt: new Date(),
}).where(eq(documents.id,
params.documentId));

return NextResponse.json({
    success: true,
    nodeCount: concepts.length,

```



```
    edgeCount: relationships.length,  
    flashcardCount,  
  });  
}
```

/app/api/graph/[documentId]/route.ts

```
import { NextRequest, NextResponse } from  
"next/server";  
import { auth } from "@lib/auth";  
import { db } from "@lib/db";  
import { conceptNodes, conceptEdges } from  
"@drizzle/schema";  
import { eq } from "drizzle-orm";  
  
export async function GET(  
  req: NextRequest,  
  { params }: { params: { documentId: string }  
}  
) {  
  const session = await auth.api.getSession({  
    headers: req.headers });  
  if (!session) return NextResponse.json({  
    error: "Unauthorized" }, { status: 401 });  
  
  const nodes = await  
    db.select().from(conceptNodes)  
      .where(eq(conceptNodes.documentId,  
        params.documentId));  
  const edges = await  
    db.select().from(conceptEdges)  
      .where(eq(conceptEdges.documentId,  
        params.documentId));
```

```
    return NextResponse.json({ nodes, edges });
  }
```

/app/api/graph/[documentId]/personalised/route.ts

```
import { NextRequest, NextResponse } from
"next/server";
import { auth } from "@lib/auth";
import { db } from "@lib/db";
import { conceptNodes, userProgress } from
"@drizzle/schema";
import { eq, and } from "drizzle-orm";

export async function GET(
  req: NextRequest,
  { params }: { params: { documentId: string } }
) {
  const session = await auth.api.getSession({
    headers: req.headers });
  if (!session) return NextResponse.json({
    error: "Unauthorized" }, { status: 401 });

  const nodes = await
    db.select().from(conceptNodes)
      .where(eq(conceptNodes.documentId,
        params.documentId));

  const progress = await
    db.select().from(userProgress)
      .where(
        and(
          eq(userProgress.documentId,
            params.documentId),
          eq(userProgress.userId,
```

```

    session.user.id)
    )
    );

    const progressMap = new Map(progress.map(p =>
[p.nodeId, p]));

    const enriched = nodes.map(n => ({
      ...n,
      memoryStrength:
progressMap.get(n.id)?.memoryStrength ?? 0.5,
      visitCount:
progressMap.get(n.id)?.visitCount ?? 0,
    })).sort((a, b) => a.memoryStrength -
b.memoryStrength); // weakest first

    return NextResponse.json(enriched);
  }

```

/app/api/flashcards/[documentId]/route.ts

```

import { NextRequest, NextResponse } from
"next/server";
import { auth } from "@lib/auth";
import { db } from "@lib/db";
import { flashcards, userProgress } from
"@drizzle/schema";
import { eq, and } from "drizzle-orm";

export async function GET(
  req: NextRequest,
  { params }: { params: { documentId: string } }
) {
  const session = await auth.api.getSession({

```

```

headers: req.headers });
    if (!session) return NextResponse.json({
error: "Unauthorized" }, { status: 401 });

    const cards = await
db.select().from(flashcards)
    .where(
        and(
            eq(flashcards.documentId,
params.documentId),
            eq(flashcards.userId, session.user.id)
        )
    );

    const progress = await
db.select().from(userProgress)
    .where(
        and(
            eq(userProgress.documentId,
params.documentId),
            eq(userProgress.userId,
session.user.id)
        )
    );

    const progressMap = new Map(progress.map(p =>
[p.nodeId, p.memoryStrength]));

    // Sort weakest memory first
(personalisation)
    const sorted = cards.sort((a, b) => {
        const aStrength = progressMap.get(a.nodeId)
        ?? 0.5;
        const bStrength = progressMap.get(b.nodeId)
        ?? 0.5;
        return aStrength - bStrength;
    });

```

```
});

return NextResponse.json(sorted);
}
```

`/app/api/flashcards/[id]/answer/route.ts`

```
import { NextRequest, NextResponse } from
"next/server";
import { auth } from "@lib/auth";
import { db } from "@lib/db";
import { flashcards, userProgress } from
"@drizzle/schema";
import { eq, and } from "drizzle-orm";

function calculateMemoryStrength(
  correctAnswers: number,
  totalAttempts: number,
  lastReviewedAt: Date | null
): number {
  const baseAccuracy = totalAttempts > 0 ?
correctAnswers / totalAttempts : 0.5;

  let recencyBoost = 1.0;
  if (lastReviewedAt) {
    const daysSince = (Date.now() -
lastReviewedAt.getTime()) / (1000 * 60 * 60 *
24);
    if (daysSince > 7) recencyBoost = 0.70;
    else if (daysSince > 3) recencyBoost =
0.85;
    else if (daysSince > 1) recencyBoost =
0.95;
  }
}
```

```

    return Math.min(1, Math.max(0, baseAccuracy *
0.7 + recencyBoost * 0.3));
}

export async function POST(
  req: NextRequest,
  { params }: { params: { id: string } }
) {
  const session = await auth.api.getSession({
headers: req.headers });
  if (!session) return NextResponse.json({
error: "Unauthorized" }, { status: 401 });

  const { correct, timeSpent } = await
req.json();

  const [card] = await
db.select().from(flashcards).where(eq(flashcard
s.id, params.id));
  if (!card) return NextResponse.json({ error:
"Not found" }, { status: 404 });

  // Upsert progress
  const [existing] = await
db.select().from(userProgress)
    .where(and(
      eq(userProgress.userId, session.user.id),
      eq(userProgress.nodeId, card.nodeId)
    ));

  const correctAnswers =
(existing?.correctAnswers ?? 0) + (correct ? 1
: 0);
  const totalAttempts =
(existing?.totalAttempts ?? 0) + 1;
  const timeSpentTotal =

```

```

(existing?.timeSpentSeconds ?? 0) + (timeSpent
?? 0);
    const memoryStrength =
calculateMemoryStrength(correctAnswers,
totalAttempts, existing?.lastReviewedAt ??
null);

    if (existing) {
        await db.update(userProgress)
            .set({
                correctAnswers,
                totalAttempts,
                timeSpentSeconds: timeSpentTotal,
                memoryStrength,
                visitCount: (existing.visitCount ?? 0)
+ 1,
                lastReviewedAt: new Date(),
                updatedAt: new Date(),
            })
            .where(eq(userProgress.id, existing.id));
    } else {
        await db.insert(userProgress).values({
            userId: session.user.id,
            nodeId: card.nodeId,
            documentId: card.documentId,
            correctAnswers,
            totalAttempts,
            timeSpentSeconds: timeSpentTotal,
            memoryStrength,
            visitCount: 1,
            lastReviewedAt: new Date(),
        });
    }

    return NextResponse.json({ memoryStrength,

```

```
correct });  
}
```

/app/api/progress/[documentId]/route.ts

```
import { NextRequest, NextResponse } from  
"next/server";  
import { auth } from "@lib/auth";  
import { db } from "@lib/db";  
import { userProgress, conceptNodes } from  
"@drizzle/schema";  
import { eq, and } from "drizzle-orm";  
  
export async function GET(  
  req: NextRequest,  
  { params }: { params: { documentId: string }  
}  
) {  
  const session = await auth.api.getSession({  
    headers: req.headers });  
  if (!session) return NextResponse.json({  
    error: "Unauthorized" }, { status: 401 });  
  
  const progress = await  
    db.select().from(userProgress)  
      .where(and(  
        eq(userProgress.documentId,  
          params.documentId),  
        eq(userProgress.userId, session.user.id)  
      ));  
  
  const nodes = await  
    db.select().from(conceptNodes)  
      .where(eq(conceptNodes.documentId,  
        params.documentId));
```



```

    const nodeMap = new Map(nodes.map(n => [n.id,
n.label]));

    const enriched = progress.map(p => ({
      ...p,
      nodeLabel: nodeMap.get(p.nodeId) ??
"Unknown",
    }));

    return NextResponse.json(enriched);
  }

```

/app/api/progress/node/[nodeId]/route.ts

```

import { NextRequest, NextResponse } from
"next/server";
import { auth } from "@lib/auth";
import { db } from "@lib/db";
import { userProgress, conceptNodes } from
"@drizzle/schema";
import { eq, and } from "drizzle-orm";

export async function POST(
  req: NextRequest,
  { params }: { params: { nodeId: string } }
) {
  const session = await auth.api.getSession({
headers: req.headers });
  if (!session) return NextResponse.json({
error: "Unauthorized" }, { status: 401 });

  const { timeSpent } = await req.json();

  const [node] = await

```

```

db.select().from(conceptNodes).where(eq(concept
Nodes.id, params.nodeId));
  if (!node) return NextResponse.json({ error:
"Not found" }, { status: 404 });

  const [existing] = await
db.select().from(userProgress)
  .where(and(
    eq(userProgress.userId, session.user.id),
    eq(userProgress.nodeId, params.nodeId)
  ));

  if (existing) {
    await db.update(userProgress).set({
      visitCount: (existing.visitCount ?? 0) +
1,
      timeSpentSeconds:
(existing.timeSpentSeconds ?? 0) + (timeSpent
?? 0),
      updatedAt: new Date(),
    }).where(eq(userProgress.id, existing.id));
  } else {
    await db.insert(userProgress).values({
      userId: session.user.id,
      nodeId: params.nodeId,
      documentId: node.documentId,
      visitCount: 1,
      timeSpentSeconds: timeSpent ?? 0,
    });
  }

  return NextResponse.json({ success: true });
}

```

`/app/api/progress/summary/route.ts`

```

import { NextRequest, NextResponse } from
"next/server";
import { auth } from "@lib/auth";
import { db } from "@lib/db";
import { userProgress, conceptNodes } from
"@drizzle/schema";
import { eq } from "drizzle-orm";
import { getSuggestions } from "@lib/gemini";

export async function GET(req: NextRequest) {
  const session = await auth.api.getSession({
    headers: req.headers });
  if (!session) return NextResponse.json({
    error: "Unauthorized" }, { status: 401 });

  const progress = await
db.select().from(userProgress)
  .where(eq(userProgress.userId,
session.user.id));

  const nodeIds = [...new Set(progress.map(p =>
p.nodeId))];
  const nodes = await
db.select().from(conceptNodes)
  .where(eq(conceptNodes.documentId,
progress[0]?.documentId ?? ""));

  const nodeMap = new Map(nodes.map(n => [n.id,
n.label]));
  const scores: Record<string, number> = {};
  for (const p of progress) {
    const label = nodeMap.get(p.nodeId);
    if (label) scores[label] = p.memoryStrength
?? 0.5;
  }
}

```

```
const suggestions = await
getSuggestions(scores);
return NextResponse.json({ suggestions,
scores });
}
```

PART 3 — FRONTEND LAYER

All Pages + Components

3A. FILE: `/app/layout.tsx`

```
import type { Metadata } from "next";
import { Space_Grotesk, Inter, Fira_Code } from
"next/font/google";
import "./globals.css";

const spaceGrotesk = Space_Grotesk({
  subsets: ["latin"],
  weight: ["400", "500", "600", "700"],
  variable: "--font-space-grotesk",
});
const inter = Inter({ subsets: ["latin"],
variable: "--font-inter" });
const firaCode = Fira_Code({ subsets:
["latin"], weight: ["400"], variable: "--font-
fira-code" });

export const metadata: Metadata = {
  title: "Mnemo — Remember Everything",
  description: "Transform any lecture into a 3D
```

```

knowledge universe",
};

export default function RootLayout({ children
}: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body className=
`${spaceGrotesk.variable} ${inter.variable}
${firaCode.variable} bg-[#050510] text-
[#F0F0FF]`>
        {children}
      </body>
    </html>
  );
}

```

3B. FILE: `/app/globals.css`

```

@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  --bg-primary:    #050510;
  --bg-secondary:  #0A0A1F;
  --bg-tertiary:   #0F0F2E;
  --accent-violet: #6C63FF;
  --accent-cyan:   #00D4FF;
  --accent-glow:   #A78BFA;
  --text-primary:  #F0F0FF;
  --text-muted:    #8888AA;
  --border:        #1E1E3F;
}

```

```

}

body {
  font-family: var(--font-inter), sans-serif;
  background: var(--bg-primary);
  color: var(--text-primary);
}

h1, h2, h3, h4 {
  font-family: var(--font-space-grotesk), sans-serif;
}

.font-mono {
  font-family: var(--font-fira-code),
monospace;
}

/* Subtle grid overlay on all pages */
body::before {
  content: "";
  position: fixed;
  inset: 0;
  background-image:
    linear-gradient(rgba(108, 99, 255, 0.03)
1px, transparent 1px),
    linear-gradient(90deg, rgba(108, 99, 255,
0.03) 1px, transparent 1px);
  background-size: 40px 40px;
  pointer-events: none;
  z-index: 0;
}

```

3C. FILE: `/next.config.js`

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    serverComponentsExternalPackages: ["pdf-
parse"],
  },
};

module.exports = nextConfig;
```

3D. FILE: `/components/layout/Sidebar.tsx`

```
"use client";
import { motion } from "framer-motion";
import Link from "next/link";
import { usePathname } from "next/navigation";
import { BookOpen, Upload, BarChart3, Home,
Logout } from "lucide-react";
import { signOut, useSession } from
"@/lib/auth-client";

const navLinks = [
  { href: "/", label: "Dashboard", icon: Home
},
  { href: "/upload", label: "Upload", icon:
Upload },
  { href: "/progress", label: "Progress", icon:
BarChart3 },
];

export function Sidebar() {
  const pathname = usePathname();
  const { data: session } = useSession();
```

```

return (
  <motion.aside
    initial={{ x: -64, opacity: 0 }}
    animate={{ x: 0, opacity: 1 }}
    transition={{ duration: 0.4 }}
    className="w-64 h-screen bg-[#0A0A1F]/80
backdrop-blur-xl border-r border-[#1E1E3F] flex
flex-col fixed left-0 top-0 z-50"
  >
    {/* Logo */}
    <div className="p-6 border-b border-
[#1E1E3F]">
      <h1 className="text-2xl font-bold text-
[#6C63FF] font-[Space_Grotesk]">
        mnemo
      </h1>
      <p className="text-xs text-[#8888AA]
mt-1">remember everything</p>
    </div>

    {/* Nav */}
    <nav className="flex-1 p-4 space-y-1">
      {navLinks.map(({ href, label, icon:
Icon }) => {
        const active = pathname === href;
        return (
          <Link
            key={href}
            href={href}
            className={`flex items-center
gap-3 px-3 py-2.5 rounded-lg transition-all ${
active
          ? "bg-[#6C63FF]/20 text-
[#A78BFA] border-l-2 border-[#6C63FF]"
          : "text-[#8888AA] hover:bg-

```



```

[#0F0F2E] hover:text-[#F0F0FF]"
    }`}
    >
    <Icon size={18} />
    <span className="text-sm font-
medium">{label}</span>
  </Link>
);
}}
</nav>

{/* User info */}
{session?.user && (
  <div className="p-4 border-t border-
[#1E1E3F]">
    <div className="flex items-center
gap-3 mb-3">
      <div className="w-8 h-8 rounded-
full bg-[#6C63FF]/30 flex items-center justify-
center text-sm font-bold text-[#A78BFA]">
        {session.user.name?.
[0]?.toUpperCase()}
      </div>
      <div>
        <p className="text-sm font-medium
text-[#F0F0FF] truncate">{session.user.name}
        </p>
        <p className="text-xs text-
[#8888AA] truncate">{session.user.email}</p>
      </div>
    </div>
    <button
      onClick={() => signOut()}
      className="w-full flex items-center
gap-2 px-3 py-2 text-xs text-[#8888AA]
hover:text-red-400 hover:bg-red-400/10 rounded-

```

```

lg transition-all"
    >
      <LogOut size={14} />
      Sign out
    </button>
  </div>
  )}
</motion.aside>
);
}

```

3E. FILE: /app/(dashboard)/layout.tsx

```

import { Sidebar } from
"@/components/layout/Sidebar";

export default function DashboardLayout({
  children }: { children: React.ReactNode }) {
  return (
    <div className="flex min-h-screen">
      <Sidebar />
      <main className="flex-1 ml-64 p-8 min-h-
screen overflow-y-auto">
        {children}
      </main>
    </div>
  );
}

```

3F. FILE: /app/(dashboard)/page.tsx

```

"use client";
import { useEffect, useState } from "react";
import { motion } from "framer-motion";
import { DocumentCard } from
"@/components/ui/DocumentCard";
import Link from "next/link";
import { Upload, Brain } from "lucide-react";

interface Document {
  id: string;
  title: string;
  subject: string | null;
  masteryScore: number | null;
  totalNodes: number | null;
  status: string | null;
  createdAt: string;
}

export default function Dashboard() {
  const [docs, setDocs] = useState<Document[]>
([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("/api/documents")
      .then(r => r.json())
      .then(d => { setDocs(d);
setLoading(false); });
  }, []);

  return (
    <div>
      {/* Hero */}
      <motion.div
        initial={{ y: 20, opacity: 0 }}
        animate={{ y: 0, opacity: 1 }}

```

```

        transition={{ duration: 0.4 }}
        className="mb-8"
      >
        <h1 className="text-4xl font-bold text-
[#F0F0FF]">Your Knowledge Universe</h1>
        <p className="text-[#8888AA] mt-
2">Upload a lecture. Watch it become a 3D
memory map.</p>
      </motion.div>

      { /* Upload CTA */ }
      <motion.div
        initial={{ y: 20, opacity: 0 }}
        animate={{ y: 0, opacity: 1 }}
        transition={{ duration: 0.4, delay: 0.1
}}
        className="mb-8"
      >
        <Link
          href="/upload"
          className="flex items-center gap-4 p-
6 rounded-xl border border-dashed border-
[#6C63FF]/50 hover:border-[#6C63FF] hover:bg-
[#6C63FF]/5 transition-all group"
        >
          <div className="w-12 h-12 rounded-xl
bg-[#6C63FF]/20 flex items-center justify-
center group-hover:bg-[#6C63FF]/30 transition-
all">
            <Upload size={24} className="text-
[#6C63FF]" />
          </div>
          <div>
            <p className="font-semibold text-
[#F0F0FF]">Upload a new lecture</p>
            <p className="text-sm text-

```

```

[#8888AA]">PDF – Gemini will map it into a 3D
knowledge graph</p>
</div>
</Link>
</motion.div>

{/* Document Grid */}
{loading ? (
  <div className="grid grid-cols-1
md:grid-cols-2 lg:grid-cols-3 gap-4">
    {[...Array(3)].map((_, i) => (
      <div key={i} className="h-48
rounded-xl bg-[#0A0A1F] animate-pulse" />
    ))}
  </div>
) : docs.length === 0 ? (
  <motion.div
    initial={{ opacity: 0 }}
    animate={{ opacity: 1 }}
    className="text-center py-24"
  >
    <Brain size={48} className="text-
[#1E1E3F] mx-auto mb-4" />
    <p className="text-[#8888AA]">No
lectures yet. Upload your first one above.</p>
  </motion.div>
) : (
  <motion.div
    initial="hidden"
    animate="visible"
    variants={{ visible: { transition: {
staggerChildren: 0.08 } } }}
    className="grid grid-cols-1 md:grid-
cols-2 lg:grid-cols-3 gap-4"
  >
    {docs.map(doc => (

```

```

        <DocumentCard key={doc.id} doc=
{doc} />
      )}}
    </motion.div>
  )}
</div>
);
}

```

3G. FILE: `/components/ui/DocumentCard.tsx`

```

"use client";
import { motion } from "framer-motion";
import Link from "next/link";
import { Telescope, BookOpen } from "lucide-react";

interface Props {
  doc: {
    id: string;
    title: string;
    subject: string | null;
    masteryScore: number | null;
    totalNodes: number | null;
    status: string | null;
  };
}

export function DocumentCard({ doc }: Props) {
  const mastery = doc.masteryScore ?? 0;
  const circumference = 2 * Math.PI * 20;
  const strokeDashoffset = circumference -
(mastery / 100) * circumference;

```

```

return (
  <motion.div
    variants={{ hidden: { y: 20, opacity: 0
}, visible: { y: 0, opacity: 1 } }}
    whileHover={{ scale: 1.02 }}
    className="bg-[#0A0A1F] border border-
[#1E1E3F] rounded-xl p-5 hover:border-
[#6C63FF]/50 hover:shadow-
[0_0_20px_rgba(108,99,255,0.1)] transition-all
cursor-pointer"
  >
    <div className="flex items-start justify-
between mb-4">
      <div className="flex-1 mr-4">
        <h3 className="font-semibold text-
[#F0F0FF] line-clamp-2">{doc.title}</h3>
        {doc.subject && (
          <span className="inline-block mt-1
px-2 py-0.5 text-xs rounded-full bg-
[#00D4FF]/10 text-[#00D4FF] border border-
[#00D4FF]/20">
            {doc.subject}
          </span>
        )}
      </div>
      {/* Mastery ring */}
      <svg width="52" height="52" viewBox="0
0 52 52">
        <circle cx="26" cy="26" r="20"
fill="none" stroke="#1E1E3F" strokeWidth="3" />
        <circle
          cx="26" cy="26" r="20" fill="none"
          stroke="#6C63FF" strokeWidth="3"
          strokeDasharray={circumference}
          strokeDashoffset={strokeDashoffset}

```

```

        strokeLinecap="round"
        transform="rotate(-90 26 26)"
    />
    <text x="26" y="30"
textAnchor="middle" fontSize="10"
fill="#A78BFA" fontFamily="Space Grotesk">
        {Math.round(mastery)}%
    </text>
</svg>
</div>
<p className="text-xs text-[#8888AA] mb-
4">
    {doc.totalNodes ?? 0} concept nodes
</p>
<div className="flex gap-2">
    <Link
        href={` /universe/${doc.id}`}
        className="flex-1 flex items-center
justify-center gap-1.5 py-2 text-sm bg-
[#6C63FF] hover:bg-[#5B52E8] text-white
rounded-lg transition-all"
    >
        <Telescope size={14} />
        Open Universe
    </Link>
    <Link
        href={` /flashcards/${doc.id}`}
        className="flex items-center justify-
center px-3 py-2 text-sm border border-
[#1E1E3F] hover:border-[#6C63FF]/50 text-
[#8888AA] hover:text-[#F0F0FF] rounded-lg
transition-all"
    >
        <BookOpen size={14} />
    </Link>
</div>

```



```
    </motion.div>
  );
}
```

3H. FILE: `/app/upload/page.tsx`

```
"use client";
import { useState, useCallback } from "react";
import { motion, AnimatePresence } from
"framer-motion";
import { useRouter } from "next/navigation";
import { Upload, Check, Loader2 } from "lucide-
react";

const STEPS = ["Extracting Text", "Mapping
Concepts", "Generating Flashcards",
"Complete"];

export default function UploadPage() {
  const router = useRouter();
  const [file, setFile] = useState<File | null>
(null);
  const [dragActive, setDragActive] =
useState(false);
  const [step, setStep] = useState(-1); // -1 =
not started
  const [error, setError] = useState("");

  const handleDrop = useCallback((e:
React.DragEvent) => {
    e.preventDefault();
    setDragActive(false);
    const dropped = e.dataTransfer.files[0];
```

```

    if (dropped?.type === "application/pdf")
      setFile(dropped);
    }, []);

    const handleUpload = async () => {
      if (!file) return;
      setError("");
      setStep(0);

      const formData = new FormData();
      formData.append("file", file);
      formData.append("title",
file.name.replace(".pdf", ""));

      try {
        const uploadRes = await
fetch("/api/documents/upload", { method:
"POST", body: formData });
        if (!uploadRes.ok) throw new
Error("Upload failed");
        const doc = await uploadRes.json();

        setStep(1);
        const processRes = await
fetch(`/api/process/${doc.id}`, { method:
"POST" });
        setStep(2);
        if (!processRes.ok) throw new
Error("Processing failed");
        setStep(3);

        setTimeout(() =>
router.push(`/universe/${doc.id}`), 1200);
      } catch (err: any) {
        setError(err.message ?? "Something went
wrong. Try again.");
      }
    }
  }
}

```

```

        setStep(-1);
    }
};

return (
  <div className="max-w-xl mx-auto pt-12">
    <motion.h1
      initial={{ y: 20, opacity: 0 }}
      animate={{ y: 0, opacity: 1 }}
      className="text-3xl font-bold mb-2"
    >
      Upload a Lecture
    </motion.h1>
    <p className="text-[#8888AA] mb-8">PDF
only – Gemini will map it into your
universe</p>

    { /* Drop Zone */ }
    <motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      onDragOver={e => { e.preventDefault();
setDragActive(true); }}
      onDragLeave={() =>
setDragActive(false)}
      onDrop={handleDrop}
      className={`border-2 border-dashed
rounded-xl p-12 text-center transition-all
cursor-pointer ${
        dragActive
          ? "border-[#6C63FF] bg-
[#6C63FF]/10"
          : file
          ? "border-[#6BCB77] bg-[#6BCB77]/5"
          : "border-[#1E1E3F] hover:border-
[#6C63FF]/50"
      }}
    >

```

```

    }}
    onClick={() =>
document.getElementById("file-input")?.click()}
  >
    <input
      id="file-input"
      type="file"
      accept="application/pdf"
      className="hidden"
      onChange={e =>
setFile(e.target.files?.[0] ?? null)}
    />
    <Upload size={40} className={`mx-auto
mb-3 ${file ? "text-[#6BCB77]" : "text-
[#8888AA]"}`} />
    {file ? (
      <p className="text-[#F0F0FF] font-
medium">{file.name}</p>
    ) : (
      <p className="text-[#8888AA]">Drop
your PDF here or click to browse</p>
    )}
  </motion.div>

  {/* Upload Button */}
  {file && step === -1 && (
    <motion.button
      initial={{ y: 10, opacity: 0 }}
      animate={{ y: 0, opacity: 1 }}
      onClick={handleUpload}
      className="w-full mt-4 py-3 bg-
[#6C63FF] hover:bg-[#5B52E8] text-white font-
semibold rounded-xl transition-all"
    >
      Build Knowledge Universe
    </motion.button>
  )}

```

```

    })

    { /* Processing Steps */
    <AnimatePresence>
      {step >= 0 && (
        <motion.div
          initial={{ opacity: 0, y: 20 }}
          animate={{ opacity: 1, y: 0 }}
          className="mt-6 space-y-3"
          >
            {STEPS.map((s, i) => (
              <div key={s} className="flex
items-center gap-3">
                <div className={`w-6 h-6
rounded-full flex items-center justify-center
transition-all ${
                  i < step ? "bg-[#6BCB77]" :
                  i === step ? "bg-[#6C63FF]" :
                  "bg-[#1E1E3F]"
                }`}>
                  {i < step ? <Check size={12}
/> : i === step ? <Loader2 size={12}
className="animate-spin" /> : null}
                </div>
                <span className={`text-sm ${i
<= step ? "text-[#F0F0FF]" : "text-
[#8888AA]"}`}>{s}</span>
              </div>
            ))}
          </motion.div>
        )}
      </AnimatePresence>

      {error && (
        <motion.p initial={{ opacity: 0 }}
          animate={{ opacity: 1 }} className="mt-4 text-

```

```

red-400 text-sm">
    {error}
  </motion.p>
  })
</div>
);
}

```

31. FILE: /components/3d/KnowledgeUniverse.tsx

```

"use client";
import { Canvas } from "@react-three/fiber";
import { OrbitControls, Stars } from "@react-three/drei";
import { ConceptNode } from "../ConceptNode";
import { ConceptEdge } from "../ConceptEdge";

interface Node {
  id: string;
  label: string;
  explanation: string;
  importance: number;
  positionX: number;
  positionY: number;
  positionZ: number;
  memoryStrength?: number;
}

interface Edge {
  id: string;
  sourceNodeId: string;
  targetNodeId: string;
  strength: number;
}

```

```

}

interface Props {
  nodes: Node[];
  edges: Edge[];
  onNodeClick: (node: Node) => void;
}

export function KnowledgeUniverse({ nodes,
edges, onNodeClick }: Props) {
  return (
    <Canvas
      style={{ width: "100vw", height: "100vh"
    }}
    camera={{ position: [0, 0, 18], fov: 60
    }}
    >
      <ambientLight intensity={0.3} />
      <pointLight position={[10, 10, 10]}
intensity={0.8} color="#6C63FF" />
      <Stars count={3000} depth={50} factor={4}
fade />
      <OrbitControls enableDamping
dampingFactor={0.05} maxDistance={40} />

      {edges.map(edge => {
        const src = nodes.find(n => n.id ===
edge.sourceNodeId);
        const tgt = nodes.find(n => n.id ===
edge.targetNodeId);
        if (!src || !tgt) return null;
        return (
          <ConceptEdge
            key={edge.id}
            start={[src.positionX,
src.positionY, src.positionZ]}

```

```

        end={[tgt.positionX, tgt.positionY,
tgt.positionZ]}
        strength={edge.strength}
      />
    );
  }}

  {nodes.map(node => (
    <ConceptNode
      key={node.id}
      node={node}
      memoryStrength={node.memoryStrength
?? 0.5}
      onClick={() => onNodeClick(node)}
    />
  ))}
</Canvas>
);
}

```

3J. FILE: `/components/3d/ConceptNode.tsx`

```

"use client";
import { useRef, useState } from "react";
import { useFrame } from "@react-three/fiber";
import { Html } from "@react-three/drei";
import * as THREE from "three";

interface Props {
  node: {
    label: string;
    importance: number;
    positionX: number;
  };
}

```



```

        positionY: number;
        positionZ: number;
    };
    memoryStrength: number;
    onClick: () => void;
}

function getGlowColor(strength: number): string
{
    if (strength < 0.3) return "#FF6B6B";
    if (strength < 0.6) return "#FFD93D";
    return "#6BCB77";
}

export function ConceptNode({ node,
memoryStrength, onClick }: Props) {
    const meshRef = useRef<THREE.Mesh>(null);
    const [hovered, setHovered] =
useState(false);
    const radius = (node.importance / 10) * 0.8 +
0.3;
    const glowColor =
getGlowColor(memoryStrength);

    useFrame(() => {
        if (meshRef.current) {
            const target = hovered ? 1.15 : 1.0;
            meshRef.current.scale.setScalar(

THREE.MathUtils.lerp(meshRef.current.scale.x,
target, 0.1)
            );
        }
    });

    return (

```

```

    <group position={[node.positionX,
node.positionY, node.positionZ]}>
      <mesh
        ref={meshRef}
        onClick={{(e) => { e.stopPropagation();
onClick(); }}}
        onPointerOver={() => {
setHovered(true); document.body.style.cursor =
"pointer"; }}
        onPointerOut={() => {
setHovered(false); document.body.style.cursor =
"auto"; }}
      >
        <sphereGeometry args={[radius, 32, 32]}
/>
        <meshStandardMaterial
          color={glowColor}
          emissive={glowColor}
          emissiveIntensity={hovered ? 1.2 :
0.4}
          roughness={0.2}
          metalness={0.3}
        />
      </mesh>
      <Html
        distanceFactor={12}
        style={{
          pointerEvents: "none",
          color: "#F0F0FF",
          fontFamily: "Space Grotesk, sans-
serif",
          fontSize: "11px",
          fontWeight: 600,
          whiteSpace: "nowrap",
          textShadow: `0 0 8px ${glowColor}`,
          transform: "translateY(-28px)",

```

```

    }}
  >
    {node.label}
  </Html>
</group>
);
}

```

3K. FILE: `/components/3d/ConceptEdge.tsx`

```

"use client";
import { Line } from "@react-three/drei";

interface Props {
  start: [number, number, number];
  end: [number, number, number];
  strength: number;
}

export function ConceptEdge({ start, end,
  strength }: Props) {
  return (
    <Line
      points={[start, end]}
      color="#00D4FF"
      lineWidth={strength * 1.5}
      transparent
      opacity={0.3 + strength * 0.3}
    />
  );
}

```

3L. FILE: /app/universe/[documentId]/page.tsx

```
"use client";
import { useEffect, useState } from "react";
import { useParams, useRouter } from
"next/navigation";
import dynamic from "next/dynamic";
import { motion, AnimatePresence } from
"framer-motion";
import { ArrowLeft, BookOpen } from "lucide-
react";
import { NodeDetailPanel } from
"@/components/ui/NodeDetailPanel";

// Dynamic import to avoid SSR for Three.js
const KnowledgeUniverse = dynamic(
  () =>
import("@/components/3d/KnowledgeUniverse").the
n(m => m.KnowledgeUniverse),
  { ssr: false }
);

export default function UniversePage() {
  const { documentId } = useParams();
  const router = useRouter();
  const [graphData, setGraphData] = useState<{
nodes: any[]; edges: any[] } | null>(null);
  const [selectedNode, setSelectedNode] =
useState<any | null>(null);
  const [docTitle, setDocTitle] =
useState("Knowledge Universe");

  useEffect(() => {
    Promise.all([

fetch(`/api/graph/${documentId}/personalised`).
```

```

then(r => r.json()),
    fetch(`/api/graph/${documentId}`).then(r
=> r.json()),
    ]).then(([personalised, { edges }]) => {
        setGraphData({ nodes: personalised, edges
});
    });
    }, [documentId]);

    return (
        <div className="relative w-screen h-screen
overflow-hidden bg-[#050510]">
            {/* 3D Canvas */}
            {graphData && (
                <KnowledgeUniverse
                    nodes={graphData.nodes}
                    edges={graphData.edges}
                    onNodeClick={setSelectedNode}
                />
            )}

            {/* Floating Top Bar */}
            <motion.div
                initial={{ y: -20, opacity: 0 }}
                animate={{ y: 0, opacity: 1 }}
                className="absolute top-4 left-4 right-
4 flex items-center justify-between z-10"
            >
                <button
                    onClick={() => router.push("/") }
                    className="flex items-center gap-2
px-4 py-2 bg-[#0A0A1F]/80 backdrop-blur border
border-[#1E1E3F] rounded-xl text-sm text-
[#8888AA] hover:text-[#F0F0FF] transition-all"
                >
                    <ArrowLeft size={14} /> Dashboard

```

```

        </button>
        <div className="px-4 py-2 bg-
[#0A0A1F]/80 backdrop-blur border border-
[#1E1E3F] rounded-xl">
            <span className="text-sm font-
semibold text-[#F0F0FF]">{docTitle}</span>
        </div>
        <button
            onClick={() =>
router.push(`/flashcards/${documentId}`)}
            className="flex items-center gap-2
px-4 py-2 bg-[#6C63FF]/80 backdrop-blur border
border-[#6C63FF] rounded-xl text-sm text-white
hover:bg-[#6C63FF] transition-all"
        >
            <BookOpen size={14} /> Flashcards
        </button>
    </motion.div>

    {/* Node Detail Panel */}
    <AnimatePresence>
        {selectedNode && (
            <NodeDetailPanel
                node={selectedNode}
                documentId={documentId as string}
                onClose={() =>
setSelectedNode(null)}
            />
        )}
    </AnimatePresence>
</div>
);
}

```

3M. FILE: /components/ui/NodeDetailPanel.tsx

```
"use client";
import { motion } from "framer-motion";
import { X, BookOpen, Clock } from "lucide-react";
import { useRouter } from "next/navigation";

function MemoryBar({ strength }: { strength: number }) {
  const color = strength < 0.3 ? "#FF6B6B" :
  strength < 0.6 ? "#FFD93D" : "#6BCB77";
  return (
    <div className="h-2 bg-[#1E1E3F] rounded-full overflow-hidden">
      <motion.div
        initial={{ width: 0 }}
        animate={{ width: `${strength * 100}%` }}
      />
    </div>
  );
}

interface Props {
  node: { id: string; label: string;
  explanation: string; memoryStrength?: number };
  documentId: string;
  onClose: () => void;
}

export function NodeDetailPanel({ node,
  documentId, onClose }: Props) {
  const router = useRouter();
```

```

const strength = node.memoryStrength ?? 0.5;
const label = strength < 0.3 ? "Needs review"
: strength < 0.6 ? "Fading" : "Strong";

return (
  <motion.div
    initial={{ x: 400, opacity: 0 }}
    animate={{ x: 0, opacity: 1 }}
    exit={{ x: 400, opacity: 0 }}
    transition={{ type: "spring", stiffness:
300, damping: 30 }}
    className="absolute right-0 top-0 h-full
w-[380px] bg-[#0A0A1F]/95 backdrop-blur-xl
border-l border-[#1E1E3F] p-6 z-20 overflow-y-
auto"
  >
    <button
      onClick={onClose}
      className="absolute top-4 right-4 p-2
text-[#8888AA] hover:text-[#F0F0FF] rounded-lg
hover:bg-[#1E1E3F] transition-all"
    >
      <X size={16} />
    </button>

    <h2 className="text-2xl font-bold text-
[#F0F0FF] mb-2 pr-8">{node.label}</h2>
    <p className="text-sm text-[#8888AA] mb-
6">{node.explanation}</p>

    <div className="mb-6">
      <div className="flex justify-between
text-xs mb-1">
        <span className="text-
[#8888AA]">Memory Strength</span>
        <span className="text-[#A78BFA]">

```



```

{label}</span>
    </div>
    <MemoryBar strength={strength} />
  </div>

  <button
    onClick={() =>
      router.push(`/flashcards/${documentId}?
nodeId=${node.id}`)}
    className="w-full flex items-center
justify-center gap-2 py-3 bg-[#6C63FF]
hover:bg-[#5B52E8] text-white rounded-xl
transition-all text-sm font-medium"
  >
    <BookOpen size={16} />
    Practice Flashcard
  </button>
</motion.div>
);
}

```

3N. AUTH PAGES

`/app/(auth)/login/page.tsx`

```

"use client";
import { useState } from "react";
import { motion } from "framer-motion";
import Link from "next/link";
import { useRouter } from "next/navigation";
import { signIn } from "@/lib/auth-client";

export default function LoginPage() {

```

```

const router = useRouter();
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");
const [error, setError] = useState("");
const [loading, setLoading] =
useState(false);

const handleSubmit = async () => {
  setLoading(true);
  setError("");
  const res = await signIn.email({ email,
password });
  if (res.error) { setError(res.error.message
?? "Login failed"); setLoading(false); }
  else router.push("/");
};

return (
  <div className="min-h-screen flex items-
center justify-center bg-[#050510]">
    <motion.div
      initial={{ y: 20, opacity: 0 }}
      animate={{ y: 0, opacity: 1 }}
      className="w-full max-w-sm p-8 bg-
[#0A0A1F] border border-[#1E1E3F] rounded-2xl"
    >
      <h1 className="text-3xl font-bold text-
[#6C63FF] mb-1">memo</h1>
      <p className="text-[#8888AA] text-sm
mb-8">Welcome back</p>

      <div className="space-y-4">
        <input
          type="email" placeholder="Email"
value={email}
onChange={e =>

```

```

setEmail(e.target.value)}
      className="w-full px-4 py-3 bg-
[#0F0F2E] border border-[#1E1E3F] rounded-xl
text-[#F0F0FF] placeholder-[#8888AA]
focus:outline-none focus:border-[#6C63FF]
transition-all"
    />
    <input
      type="password"
placeholder="Password" value={password}
      onChange={e =>
setPassword(e.target.value)}
      className="w-full px-4 py-3 bg-
[#0F0F2E] border border-[#1E1E3F] rounded-xl
text-[#F0F0FF] placeholder-[#8888AA]
focus:outline-none focus:border-[#6C63FF]
transition-all"
    />
    {error && <p className="text-red-400
text-sm">{error}</p>}
    <motion.button
      whileHover={{ translateY: -2 }}
      onClick={handleSubmit}
      disabled={loading}
      className="w-full py-3 bg-[#6C63FF]
hover:bg-[#5B52E8] text-white font-semibold
rounded-xl transition-all disabled:opacity-50"
    >
      {loading ? "Signing in..." : "Sign
In"}
    </motion.button>
  </div>
  <p className="text-center text-
[#8888AA] text-sm mt-6">
    No account?{" "}
    <Link href="/signup" className="text-

```

```

      [#A78BFA] hover:text-[#6C63FF]">Sign up</Link>
    </p>
  </motion.div>
</div>
);
}

```

`/app/(auth)/signup/page.tsx`

```

"use client";
import { useState } from "react";
import { motion } from "framer-motion";
import Link from "next/link";
import { useRouter } from "next/navigation";
import { signUp } from "@/lib/auth-client";

export default function SignupPage() {
  const router = useRouter();
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] =
    useState(false);

  const handleSubmit = async () => {
    setLoading(true);
    setError("");
    const res = await signUp.email({ name,
    email, password });
    if (res.error) { setError(res.error.message
    ?? "Signup failed"); setLoading(false); }
    else router.push("/");
  };
}

```

```

    return (
      <div className="min-h-screen flex items-
center justify-center bg-[#050510]">
        <motion.div
          initial={{ y: 20, opacity: 0 }}
          animate={{ y: 0, opacity: 1 }}
          className="w-full max-w-sm p-8 bg-
[#0A0A1F] border border-[#1E1E3F] rounded-2xl"
        >
          <h1 className="text-3xl font-bold text-
[#6C63FF] mb-1">memo</h1>
          <p className="text-[#8888AA] text-sm
mb-8">Start remembering everything</p>

          <div className="space-y-4">
            <input type="text" placeholder="Full
name" value={name} onChange={e =>
setName(e.target.value)}
              className="w-full px-4 py-3 bg-
[#0F0F2E] border border-[#1E1E3F] rounded-xl
text-[#F0F0FF] placeholder-[#8888AA]
focus:outline-none focus:border-[#6C63FF]
transition-all" />
            <input type="email"
placeholder="Email" value={email} onChange={e
=> setEmail(e.target.value)}
              className="w-full px-4 py-3 bg-
[#0F0F2E] border border-[#1E1E3F] rounded-xl
text-[#F0F0FF] placeholder-[#8888AA]
focus:outline-none focus:border-[#6C63FF]
transition-all" />
            <input type="password"
placeholder="Password" value={password}
onChange={e => setPassword(e.target.value)}
              className="w-full px-4 py-3 bg-
[#0F0F2E] border border-[#1E1E3F] rounded-xl

```

```

text-[#F0F0FF] placeholder-[#8888AA]
focus:outline-none focus:border-[#6C63FF]
transition-all" />
      {error && <p className="text-red-400
text-sm">{error}</p>}
      <motion.button
        whileHover={{ translateY: -2 }}
        onClick={handleSubmit}
        disabled={loading}
        className="w-full py-3 bg-[#6C63FF]
hover:bg-[#5B52E8] text-white font-semibold
rounded-xl transition-all disabled:opacity-50"
      >
        {loading ? "Creating account..." :
"Create Account"}
      </motion.button>
    </div>
    <p className="text-center text-
[#8888AA] text-sm mt-6">
      Have an account?{" "}
      <Link href="/login" className="text-
[#A78BFA] hover:text-[#6C63FF]">Sign in</Link>
    </p>
  </motion.div>
</div>
);
}

```

ENVIRONMENT VARIABLES

```

# .env.local – fill these in before running

# Neon Postgres: neon.tech → New Project →

```

```
Connection String
DATABASE_URL="postgresql://..."

# Gemini API: aistudio.google.com → Get API
Key (free)
GEMINI_API_KEY="AIza..."

# Better Auth: generate any random 32+ char
string
BETTER_AUTH_SECRET="your-32-char-random-secret-
here-ok"
BETTER_AUTH_URL="http://localhost:3000"
NEXT_PUBLIC_APP_URL="http://localhost:3000"
```

DEPENDENCY INSTALL COMMANDS

Run in this exact order:

```
# 1. Create project
npx create-next-app@latest mnemo --typescript -
-tailwind --app --no-src-dir --import-alias
"@/*"
cd mnemo

# 2. Install all dependencies in ONE command
npm install framer-motion @react-three/fiber
@react-three/drei three @types/three drizzle-
orm @neondatabase/serverless drizzle-kit
better-auth pdf-parse @google/generative-ai
lucide-react class-variance-authority clsx
tailwind-merge

# 3. shadcn/ui init (choose: dark theme, slate
```

```
base, yes to CSS variables)
npx shadcn@latest init

# 4. Install shadcn components
npx shadcn@latest add button card dialog input
label progress badge tabs toast separator
avatar dropdown-menu

# 5. Push database schema (after filling in
DATABASE_URL)
npx drizzle-kit push

# 6. Run dev
npm run dev
```

KNOWN ERRORS & FIXES

Read before running. Reference if Antigravity hits issues.

Error	Fix
R3F canvas not rendering	Ensure Canvas has <code>style={{width:'100vw',height:'100vh'}}</code> – not just className
Gemini API returning non-JSON	<code>.trim()</code> before <code>JSON.parse</code> , wrap in try-catch, return <code>[]</code> on failure
pdf-parse failing	Use <code>Buffer.from(await file.arrayBuffer())</code> – check Content-Type before parsing

Error	Fix
Drizzle relations not resolving	Use <code>db.select().from(table).where()</code> pattern – avoid relational queries in Phase 1
Better Auth session missing	Use <code>auth.api.getSession({ headers: request.headers })</code> in every protected route
3D nodes overlapping	Increase position range from <code>[-5,5]</code> to <code>[-8,8]</code> in <code>extractConcepts</code>
Framer Motion hydration mismatch	Add <code>'use client'</code> at top of every animated component file
Vercel build failing on pdf-parse	Add to <code>next.config.js</code> : <code>experimental: { serverComponentsExternalPackages: ['pdf-parse'] }</code>
KnowledgeUniverse SSR error	Import with <code>dynamic(..., { ssr: false })</code> in the page that uses it
Three.js types missing	<code>npm install @types/three</code>

BUILD EXECUTION ORDER

Antigravity: follow this sequence. Do not reorder.

Phase 1 – Foundation

- Create project, install deps, folder structure, fonts, CSS vars

Phase 2 – Database + Auth

- /drizzle/schema.ts, /lib/db.ts, /lib/auth.ts, auth-client.ts
- /app/api/auth/[...all]/route.ts
- npx drizzle-kit push

Phase 3 – AI + Backend APIs

- /lib/gemini.ts, /lib/pdf-parser.ts
- /app/api/documents/upload, /api/documents, /api/process/[id]
- /app/api/graph/[id], /api/graph/[id]/personalised
- /app/api/flashcards/[id], /api/flashcards/[id]/answer
- /app/api/progress/[id], /api/progress/node/[id], /api/progress/summary

Phase 4 – UI Foundation

- /app/globals.css, /app/layout.tsx, /next.config.js
- /components/layout/Sidebar.tsx
- /app/(dashboard)/layout.tsx, /app/(dashboard)/page.tsx
- /components/ui/DocumentCard.tsx
- /app/(auth)/login/page.tsx, signup/page.tsx
- /app/upload/page.tsx

Phase 5 – 3D Universe (most important)

- /components/3d/KnowledgeUniverse.tsx
- /components/3d/ConceptNode.tsx
- /components/3d/ConceptEdge.tsx

```
→ /app/universe/[documentId]/page.tsx
→ /components/ui/NodeDetailPanel.tsx
```

Phase 6 – Flashcards + Progress

```
→ /components/ui/FlashCard.tsx
→ /app/flashcards/[documentId]/page.tsx
→ /app/progress/[documentId]/page.tsx
```

Phase 7 – Polish + Deploy

```
→ Memory decay cron, loading skeletons,
error states
→ Responsive breakpoints
→ vercel --prod
```

END OF MASTER DOCUMENT

Mnemo is built to win. Execute phases in order. Do not deviate from the stack.